

Rewriting Agent Infrastructure in Rust

Why the language that was "too hard for humans" is now ideal for the machines writing our code.

Michael Yuan, PhD

Co-founder, **GOSIM & Second State** · Maintainer, **WasmEdge**

x.com/juntao · github.com/juntao

FROM RUST... TO RUST + AI

GOSIM  **Rust.**

And it turns out the language we love most *for*
systems engineers
is the same language **AI agents work best in.**

HE SAID IT THREE YEARS AGO.



This talk is about why he was right — and what's happening because of it.

The author of new code isn't human.

2024: autocomplete — human types, model suggests, human accepts.

2025: human writes a sentence, agent writes the function, the file, the project.

2026: agent runs for hours autonomously — opening PRs, fixing CI, shipping crates.

The implication no one has stated plainly:

Human-friendliness is ~~the~~ a primary constraint.

Machine-friendliness is becoming the primary constraint.

The classic complaints about Rust...

- ▶ The borrow checker is unforgiving.
- ▶ The type system is verbose — `Result<Vec<Option<T>>, MyError>`
- ▶ No garbage collector. You think about lifetimes.
- ▶ Error handling is mandatory. Every `?` is a decision.
- ▶ The compiler is slow and pedantic.

Each of these is a complaint *from a human's perspective*.

An LLM does not get tired. An LLM does not lose patience.

...are exactly what an LLM agent wants.

Property	Cost to humans	Value to LLMs
Strict compiler	Slows iteration	Catches ~90% of hallucinations before runtime
Mandatory error handling	Verbose	Forces every failure path to be considered
No null, no exceptions	Boilerplate	Eliminates whole categories of invisible bugs
Static types everywhere	Friction	Ground truth for code completion
Single binary	No REPL	Trivial to ship and execute deterministically
No GC, no runtime	Manual memory thinking	Predictable performance for inference

The compiler is the most powerful eval harness ever shipped to AI agents.

Two buckets.

BUCKET A

You build agent infra

Rust as the substrate for
harnesses, inference engines,
CLI tools.

Runtime properties.

BUCKET B

Agents emit Rust

Rust as the *output* when LLMs
are the authors.

Authoring properties.

§3A · BUILDING AGENT INFRASTRUCTURE

6 runtime advantages

when you choose Rust as your substrate.

Milliseconds vs. seconds.

Python

```
import torch
```

2–5 seconds before doing anything useful.

Rust binary

Single binary, **milliseconds to ready**.

Multiplied by hundreds of tool calls per session.

Agents make *hundreds* of tool calls. Cold-start is wasted time, multiplied.

A.2 · FOOTPRINT

5GB → 50MB.

5GB

PYTHON ML CONTAINER

50MB

STATIC RUST BINARY

100x

SMALLER

- ▶ **Edge deployment:** EchoKit voice agent — impossible on Python.
- ▶ **Container density:** 50MB pulls in 200ms; 5GB takes 30s.

No GIL. Data races as compile errors.

Agents run UI loops, model streams, parallel tool calls, MCP servers, file watchers — **simultaneously**.

The Tokio toolbox

`JoinSet`, `mpsc`, `oneshot`,
`CancellationToken`, `select!`,
`timeout`

Cancellation is first-class

Ctrl-C tears down every in-flight task cleanly. Python cancellation is a known nightmare.

A.4 · ENERGY

74× more efficient than Python.

3–4%

VS. C (PER WORKLOAD)

74×

VS. PYTHON

50%

DATACENTER CUT (AWS
EST.)

Datacenters: ~340 TWh/yr globally. AI inference is the fastest-growing slice.
For battery-powered edge AI, joules *are* the product.

Pereira et al. 2017 · [aws/sustainability-with-rust](https://aws.amazon.com/sustainability-with-rust/)

One `.wasm`. Every device. Sandboxed.

Portability

- ✓ Linux, macOS, Windows, browsers, edge gateways, IoT — **identical bits**
- ✓ Microsecond cold-start — faster than containers by orders of magnitude
- ✓ No transitive deps — the `.wasm` is the deliverable

Safety

- ✓ **Default-deny sandbox** — module gets only what host explicitly grants
- ✓ WASI capabilities replace ambient FS, network, env access
- ✓ No `~/.aws/credentials`. No `/etc/passwd`.
No surprise `curl` to attacker.com.

Both matter for agents: portability for distribution, safety for letting the LLM *choose* what to invoke.

Best-in-class tracing, built in.

`tracing` + `tracing-subscriber` +

OpenTelemetry:

- ▶ Structured JSON logs out of the box
- ▶ Spans nest naturally across async boundaries
- ▶ Compile-time enforcement of every log site
- ▶ Zero-cost when disabled

3 AM agent meltdown

Agent ran for 6 hours. Did the wrong thing.

You need to grep the span, see the inputs, reconstruct the trace — **without restarting.**

§3B · OUTPUT OF AGENTS

4 authoring advantages

when the LLM is the author.

B.1 · COMPILER AS HALLUCINATION FILTER

The agent's eval loop, for free.

1. Generate code
2. Try to compile / run
3. Read the error ← quality of THIS step is everything
4. Fix the code
5. Goto 2

Python

`NameError` from deep in a library. Or worse: program runs, returns garbage.

Rust

"Expected `&mut`, found `&` on line 47"
+ a suggested fix.

Now: *better* at Rust than at Python.

Yes, really. Even though Python's corpus is $\sim 10\times$ larger.

- ▶ **Uniform corpus:** `cargo` layout, `rustfmt`, Clippy, CI culture — everywhere.
- ▶ **Compiler as RL signal:** compile/test pass-fail trains exceptional models (Strand-Rust-Coder, Oxen.ai GRPO).
- ▶ **Structured docs:** `rustdoc` JSON → `cargo-doc-md` → LLM context windows.

The Bitter Lesson, applied to languages: less data, cleaner data, plus a verifier — beats more data, no verifier.

B.3 · DISTRIBUTION MODEL

Binary, not source.

Not "crates.io is safer." **Rust ships binaries; Python ships source + a manifest.** The package manager runs on the user's machine — that's the attack vector.

AI infra is the most-targeted ecosystem.

- ▶ **MAR 2026** LiteLLM · `.pth` on every Python startup — SSH, AWS, GCP, K8s, `.env`
- ▶ **DEC 2024** Ultralytics YOLO · 60M downloads · cryptominer in `__init__.py`
- ▶ **DEC 2022** PyTorch / torchtriton · dependency confusion, SSH exfil via DNS
- ▶ **SEP 2025** Shai-Hulud npm worm · 500+ packages, self-replicating
- ▶ **2024+** DPRK Contagious Interview · 1,700+ packages, npm/PyPI/Go/Rust

Every attack needed `pip install`.

Python `.pth` files. `setup.py`. npm `postinstall`. They are not bugs — they are **designed behavior** the attackers invoked.

Source distribution

Package manager runs on every install, every restart, every fresh sandbox — on the user's machine.

Binary distribution

One artifact, one hash. Build risk concentrates at the maintainer, not amplified across every install.

Rustdoc > Python source.

When an agent reads Python source to learn an API, it gets mush.

When it reads `rustdoc` output, it gets **typed signatures, structured examples, trait hierarchies, explicit error types** — uniformly.

Tools: `cargo-doc-md`, `rustdoc-llms`, `rust-docs` MCP server.

Higher info density per token → more useful context fits in the window → **fewer hallucinated APIs.**

§ 4 · WHERE THIS IS HAPPENING NOW

Three frontiers.

CLI tools

Harness

Inference

Modern coding agents *shell out*.

Classic	Rust replacement	Why agents prefer it
<code>grep</code>	<code>ripgrep</code> <code>(rg)</code>	10–100× faster, respects <code>.gitignore</code> , JSON
<code>find</code>	<code>fd</code>	Saner defaults, Unicode-correct
<code>cat</code>	<code>bat</code>	Syntax-highlighted, paginated
<code>ls</code>	<code>eza</code>	Git-aware, structured output
<code>cd</code>	<code>zoxide</code>	Frecency-based
<code>diff</code>	<code>delta</code>	Side-by-side, syntax-aware

Claude Code installs `ripgrep` by default. It is part of the agent's bones.

Rust's CLI ecosystem is unfair.

Where the two buckets meet: agents *use* these tools and increasingly *write* them.

Args & UI

clap

indicatif

dialoguer

tabled

owo-colors

Config & Errors

serde

dirs

eyre

snafu

fs_err

Obs & Tests

tracing

insta (snapshot)

trycmd (CLI tests)

There is **no Python equivalent** that is anywhere near as cohesive.

The syntaqlite story.

Lalit Maganti built a SQLite formatter + linter + LSP. **Started Python-vibe-coded** → "complete spaghetti." **Threw it all away.** Rewrote in Rust with AI as autocomplete-on-steroids. **Shipped 0.1 in ~2 months.**

500+

TESTS

4

PACKAGE ECOSYSTEMS

2

EDITOR EXTENSIONS

"AI is an incredible force multiplier for implementation, but it's a dangerous substitute for design."

Compile to Wasm. Lose the matrix.

Native Rust binaries are excellent — but you still ship a release per `(OS, arch)` pair. For an agent that may run on a Mac, a Linux runner, a Windows box, an x86 gateway, an ARM64 IoT device... the matrix gets ugly.

One artifact — everywhere

- ✓ One `.wasm`, every host with a Wasm runtime
- ✓ WasmEdge, Wasmtime, the browser
- ✓ Microsecond cold-start, no host pollution

Sandboxed at runtime

- ✓ Capability-restricted FS / network / env
- ✓ The skill sees only what the agent grants it
- ✓ Critical when the LLM is choosing what to invoke

Not "use Docker to contain the mess" — "compile the mess away."

§4.2 · THE HARNESS ITSELF

Harnesses are moving to Rust.

Goose

Block / Square. Open-source, multi-provider.
Production-grade.

Claurst

Clean-room Rust reimplementation of Claude
Code.

IronClaw **NEAR AI**

Rust port of OpenClaw. Single binary · **Wasm
sandboxing.**

ZeroClaw

Pure-Rust agent runtime. Multi-channel.
GPIO/I2C.

Plus `harness`, `harness-locate`, `gate4agent` — orchestrators around all of them.

Rust AI ports — CUDA + MLX.

Project	Model	Domain	Backends
qwen3_tts_rs	Alibaba Qwen3-TTS	TTS, voice cloning, 10 langs	libtorch · MLX
qwen3_asr_rs	Alibaba Qwen3-ASR	Speech recognition	libtorch · MLX
voxtral_tts_rs	Mistral Voxtral-4B	4B-param TTS	libtorch · MLX
kitten_tts_rs	KittenTTS	15M–80M lightweight TTS	ONNX (CUDA/CoreML/...)
cohere_transcribe_rs	Cohere Transcribe (Mar '26)	14-lang ASR	libtorch · MLX
Moxin / Moly	Moxin-7B + local server	OpenAI-compatible, zero Python	wgpu · MLX

CUDA on Linux, MLX on Mac — the two GPU paths that actually matter for self-hosted AI in 2026.

Not a war. A layering.

Research + Training

Python · PyTorch · JAX
Jupyter for thinking.
Tools humans use to think.

Artifacts

ONNX · safetensors ·
GGUF · TorchScript
The contract between worlds.

Inference + Skills

Rust · CLIs · harnesses ·
edge
Tools machines use to act.

Same boundary that already exists between C++ and Python in PyTorch — just moved up the stack.

If you build agent infra (Bucket A):

- ▶ **New CLI tools → default to Rust.** `clap`, `tokio`, `tracing`, `eyre`. Ship single binaries.
- ▶ **Agent harnesses → plan for Rust.** Startup time, fearless concurrency, observability, binary distribution.
- ▶ **Inference → evaluate the Rust stack seriously.** `mistral.rs`, `Candle`, `tch-rs`, `mlx-rs`, `ort`. You may not need PyTorch at runtime.
- ▶ **Edge / skills → target Wasm.** Rust + WASI-NN beats every other deployment artifact.

If agents emit code (Bucket B):

- ▶ **Default the agent's emitted language to Rust** — for the kinds of tools where Python and Node currently dominate.
- ▶ **Treat the compiler as part of your eval stack.** The cheapest, fastest, most reliable reviewer you'll ever ship.
- ▶ **Stop apologizing for the learning curve.** Your users — agents — don't have one.

AI is autocomplete on steroids. Rust gives you exactly the substrate that division of labor needs.

ONE MORE THING

Coding-by-talking needs hardware.

VibeKeys MAX

A dedicated controller for Claude Code & agent workflows.

- ▶ Dedicated **Accept / Retry / Voice** keys
- ▶ Built-in mic for voice dictation
- ▶ TFT status display for build / suggestion state
- ▶ BLE 5.0 + Wi-Fi, OTA updates
- ▶ Open-source firmware, written in **Rust** 🦀

vibekeys.dev/vibekeys-max



Code from anywhere — desk, couch, edge of the world.

§ 7 · CLOSING

**Python is for humans.
Rust is for machines.
The machines are writing the
code now —
and running it.**

Thank you.

Questions, arguments, war stories — let's go.

Michael Yuan, PhD

Co-founder, GOSIM & Second State

x.com/juntao · github.com/juntao · vibekeys.dev